Practical-1

Code-# Simple Reflex Agent - Vacuum Cleaner World with Path Cost

```
environment = {
   'A': 'Dirty',
   'B': 'Dirty'
}

# Initial position
location = 'A'

# Path cost
path_cost = 0

def reflex_agent(location, status):
   if status == "Dirty":
      return "Suck"
   elif location == "A":
      return "Right"
   else:
      return "Left"

print("Initial Environment:", environment)

while True:
status = environment[location] // the room where the vacuum is currently located is dirty or clean.
   action = reflex_agent(location, status)

   print(f"\nVacuum is at {location}")
   print(f'Room Status : {status}")
   print(f'Action Taken: {action}")

   if action == "Suck":
      environment[location] = "Clean"
      path_cost += 1      # Cost for cleaning
```

```
    elif action == "Right":
        location = "B"
        path_cost += 1      # Cost for moving

    elif action == "Left":
        location = "A"
        path_cost += 1      # Cost for moving

    if environment["A"] == "Clean" and environment["B"] == "Clean":
        print("\nBoth rooms are clean.")
        break

print("\nFinal Environment:", environment)
print("Total Path Cost:", path_cost)
```

Output:-

```
Initial Environment: {'A': 'Dirty', 'B': 'Dirty'}

Vacuum is at A
Room Status : Dirty
Action Taken: Suck

Vacuum is at A
Room Status : Clean
Action Taken: Right

Vacuum is at B
Room Status : Dirty
Action Taken: Suck

Both rooms are clean.

Final Environment: {'A': 'Clean', 'B': 'Clean'}
Total Path Cost: 3
```

Practical-2

```
import heapq

# Goal state
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)



# Heuristic function: Manhattan Distance
def h(s):
    d = 0

    for i in range(9):
        if s[i] != 0:
            j = goal.index(s[i])

            d += abs(i // 3 - j // 3) + abs(i % 3 - j % 3)

    return d



# A* Search Algorithm
def astar(start):

    # Priority queue: (f, g, state, path)
    q = [(h(start), 0, start, [])]

    # Set of visited states
    seen = set()

    while q:

        # Get state with minimum f value
        f, g, s, path = heapq.heappop(q)
```

```
# Skip if already visited
if s in seen:
    continue

# Mark state as visited
seen.add(s)

# Check whether goal is reached
if s == goal:
    return path + [s]

# Find position of blank (0)
z = s.index(0)

# Convert position into row and column
r, c = divmod(z, 3)

# Possible moves: Down, Up, Right, Left
for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:

    nr, nc = r + dr, c + dc

    # Check whether the move is valid
    if 0 <= nr < 3 and 0 <= nc < 3:

        # Find new position
        x = nr * 3 + nc

        # Create new state
        t = list(s)

        # Swap blank with adjacent tile
        t[z], t[x] = t[x], t[z]
```

```
            # Convert list back to tuple
            t = tuple(t)

            # Calculate f = g + h and add to queue
            heapq.heappush(
                q,
                (g + 1 + h(t), g + 1, t, path + [s])
            )


# Initial state
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)


# Find solution
solution = astar(start)


# Display solution
print("Solution:")
print()

for s in solution:

    print(s[0:3])
    print(s[3:6])
    print(s[6:9])
    print()
```

Practical-3

```
def winner(board):
    lines = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in lines:
        if board[a] != " " and board[a] == board[b] == board[c]:
            return board[a]

    return None

def terminal(board):
    return winner(board) is not None or " " not in board

def utility(board):
    w = winner(board)

    if w == "X":
        return 1
    if w == "O":
        return -1

    return 0



def alphabeta(board, maximizing, alpha, beta):
    if terminal(board):
```

```
    return utility(board)

if maximizing:
    value = -float("inf")

    for i in range(9):
        if board[i] == " ":
            board[i] = "X"

            value = max(
                value,
                alphabeta(board, False, alpha, beta)
            )

            board[i] = " "

            alpha = max(alpha, value)

            if alpha >= beta:
                break

    return value

else:
    value = float("inf")

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"

            value = min(
                value,
                alphabeta(board, True, alpha, beta)
            )
```

```
            board[i] = " "

            beta = min(beta, value)

            if alpha >= beta:
                break

        return value


def best_move(board):
    best = None
    best_val = -float("inf")

    for i in range(9):
        if board[i] == " ":
            board[i] = "X"

            val = alphabeta(
                board,
                False,
                -float("inf"),
                float("inf")
            )

            board[i] = " "

            if val > best_val:
                best = i
                best_val = val

    return best, best_val


# Current Tic-Tac-Toe board
```

```
board = [
    "X", "O", "X",
    " ", "O", " ",
    " ", " ", " "
]

move, value = best_move(board)

print("Best move index:", move)
print("Minimax value:", value)
```

Practical-4

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 0:
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination, auxiliary)

    # Move the largest disk to destination
    print(f"Move disk {n} from {source} -> {destination}")

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n - 1, auxiliary, source, destination)



# Main program
n = int(input("Enter number of disks: "))

print("\nTower of Hanoi using State Space Search:")
tower_of_hanoi(n, 'A', 'B', 'C')

print("\nMinimum number of moves:", 2**n - 1)
```

Output:-

Enter number of disks: 3

State Space Search:

Move disk 1 from A -> C

Move disk 2 from A -> B

Move disk 1 from C -> B

Move disk 3 from A -> C

Move disk 1 from B -> A

Move disk 2 from B -> C

Move disk 1 from A -> C

Minimum number of moves: 7

Practical-5

```
from collections import deque

def bfs(grid, start, goal):
    rows = len(grid)
    cols = len(grid[0])

    queue = deque([start])
    visited = {start}
    parent = {start: None}

    # Up, Down, Left, Right
    directions = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    while queue:
        current = queue.popleft()
```

```
        if current == goal:
            break

        for dr, dc in directions:
            nr = current[0] + dr
            nc = current[1] + dc
            next_position = (nr, nc)

            if (0 <= nr < rows and
                0 <= nc < cols and
                grid[nr][nc] != 1 and
                next_position not in visited):

                visited.add(next_position)
                parent[next_position] = current
                queue.append(next_position)

    # If goal was not reached
    if goal not in parent:
        return None

    # Create path
    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()

    return path



# 0 = free path
```

```
# 1 = obstacle

grid = [
    [0, 0, 0, 1, 0],
    [1, 0, 0, 1, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (3, 4)

path = bfs(grid, start, goal)

if path:
    print("Shortest Path:")
    print(path)
    print("Number of moves:", len(path) - 1)
else:
    print("No path found")
```

Practical-6

```
def dfs_maze(grid, start, goal):
    rows = len(grid)
    cols = len(grid[0])

    visited = set()
    path = []

    def dfs(r, c):
        # Check boundaries
        if not (0 <= r < rows and 0 <= c < cols):
            return False

        # Check wall or visited cell
        if grid[r][c] == 1 or (r, c) in visited:
```

```
            return False

        visited.add((r, c))
        path.append((r, c))

        # Goal reached
        if (r, c) == goal:
            return True

        # Down, Right, Up, Left
        directions = [
            (1, 0),
            (0, 1),
            (-1, 0),
            (0, -1)
        ]

        for dr, dc in directions:
            if dfs(r + dr, c + dc):
                return True

        # Backtracking
        path.pop()
        return False

    return path if dfs(*start) else None


# 0 = Open path
# 1 = Wall

maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
```

```
    [1, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]

start = (0, 0)
goal = (4, 2)

path = dfs_maze(maze, start, goal)

print("DFS path:", path)
print("Path found:", path is not None)
```

Practical-7

```
# 8-Queens using Backtracking

N = 8
board = [-1] * N

def is_safe(row, col):
    for previous_row in range(row):
        previous_col = board[previous_row]

        # Same column
        if previous_col == col:
            return False

        # Same diagonal
        if abs(previous_row - row) == abs(previous_col - col):
            return False

    return True



def solve(row):
    # All queens are placed
```

```
    if row == N:
        return True

    # Try every column
    for col in range(N):
        if is_safe(row, col):
            board[row] = col

            # Place queen in next row
            if solve(row + 1):
                return True

            # Backtrack
            board[row] = -1

    return False



# Solve the problem
if solve(0):
    print("Solution found:")

    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution found")
```

Practical-8

```
def chatbot_response(message):
    text = message.lower().strip()

    if any(word in text for word in ["hello", "hi", "hey"]):
        return "Hello! How can I help you?"

    if "bfs" in text:
        return "BFS explores states level by level."

    if "dfs" in text:
        return "DFS explores deeply before backtracking."

    if "a*" in text or "astar" in text:
        return "A* uses f(n) = g(n) + h(n) to guide the search."

    if "viva" in text:
        return "Prepare definitions, algorithms, complexity and code logic."

    if "lab" in text or "experiment" in text:
        return "Please study the aim, theory, algorithm, code and output."

    return "Sorry, I do not understand that question."

print("Simple Rule-Based Chatbot")
print("Type 'bye' to exit.")

while True:
    message = input("You: ")

    if message.lower().strip() in ["bye", "exit", "quit"]:
        print("Bot: Goodbye! Have a nice day.")
        break
    print("Bot:", chatbot_response(message))
```